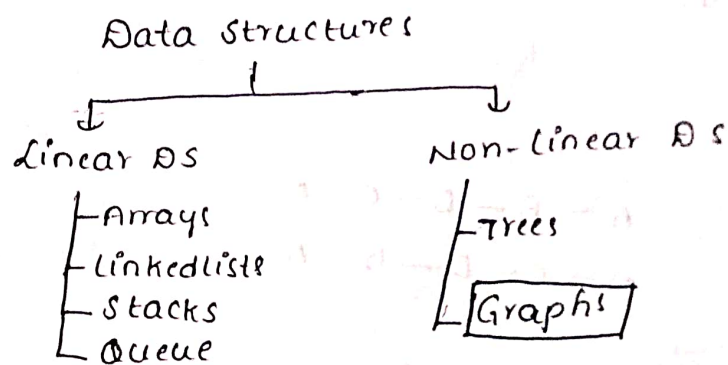


# UNIT-5

1. Graphs
2. Hashing

## GRAPHS



### Important Definitions:-

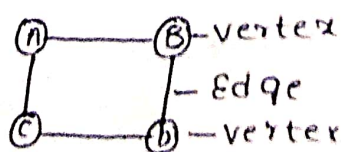
Graph: It is non-linear data structure

Graph is the combination of vertices and edges

\* Vertex → These are the values represented with circle 'O'  
Each vertex contains labels and Reference of related vertices

\* Edge → These are useful to connected related vertices  
Edge is represented with straight line '—'.

Example Graph:-

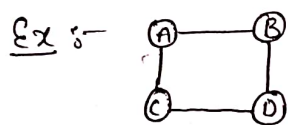


— In the above example Graph, A, B, C, D are vertices and the straight lines which connects vertices are edges.

— In Graph Data structure, the elements are inserted and deleted in Random manner.

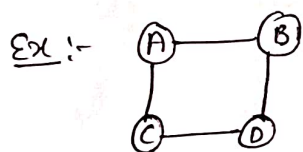
To develop and use the Graphs, we should know the some Key terms. They are:

⊗ Adjacency → If two vertices are connected by single edge then we said the two vertices are adjacent.



In the above Graph, A, B, C, D are called "Adjacent vertices".

⊗ path → It is the sequence of edges from one vertex to another vertex.

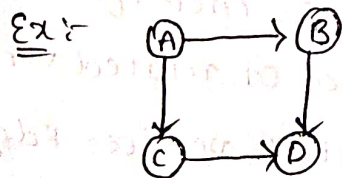


Here path form (A to A) is

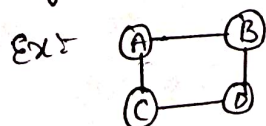
A — B — D — C — A  
A — C — D — B — A

⇒ Types of Graphs

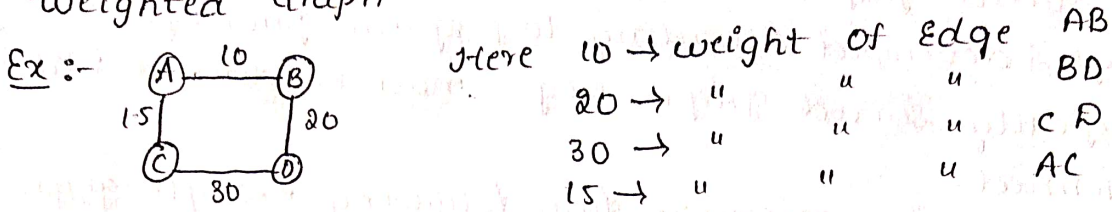
⊗ Directed Graph → A Graph with Directed Edges is called "Directed Graph". Direction represented with Arrow symbol



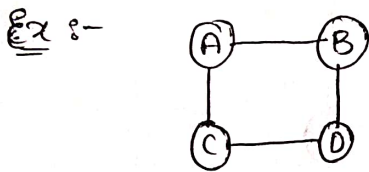
⊗ Non-Directed Graph → A Graph with non Directed Edges is called "Non-Directed Graph".



\* weighted Graph → A Graph with weighted Edges is called "weighted Graph".



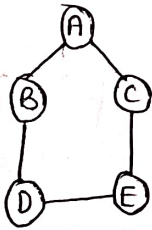
\* non-weighted graph:- A graph with non-weighted edges will be called as non-weighted graph.



\* connected & non-connected graph:-

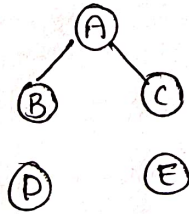
connected graph:- If any pair of vertices are connected by edges then that graph will be called "connected graph".

Ex:-



Non-connected graph:- If same pair of vertices are not connected by edges in any graph then that structure will be called as non-connected graph.

Ex:-



⇒ In a connected & non-directed graph, traversing task will be possible.

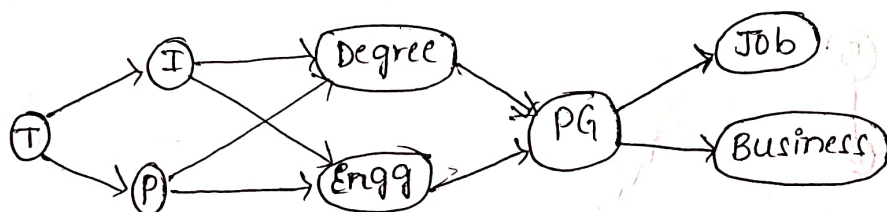
⇒ In non-connected & non-directed graph, traversing task is not possible.



\* Topologically sorted graph :- A directed graph is said to be topologically sorted graph, when all the vertices & their related edges & developed according to physical priority.

⇒ In topologically sorted graph edges must be directed & no cycles are formed.

⇒ This concept can be presented from following example graph values Degree, tenth(T), Engg, poly Technique(p), Inter(I), PG, Business, Job.

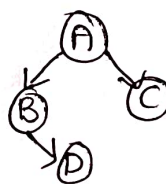
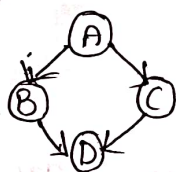


As shown above values in Graph are sorted based on their physical priorities.

\* Tree :- A graph with no cycles will be called as Tree.

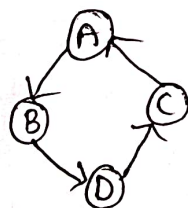
Ex:- Graph

NOT Tree



Tree.

\* Cycle :- It is nothing but a path. This path ends where it started. This will be as.

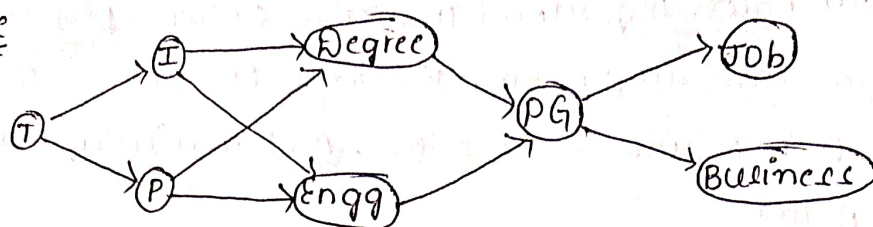


path :-  $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$ .

\* Directed Acyclic Graph (DAG) :-

The topologically sorted & directed graph will be called as directed Acyclic graph (DAG). In directed Acyclic graph no cycle will be formed then it is called Direct Acyclic Graph (DAG):

Example



Path :-

$T \rightarrow I \rightarrow \text{deg} \rightarrow PG \rightarrow \text{Job}$

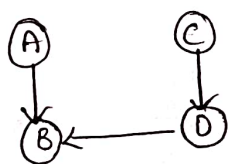
$T \rightarrow I \rightarrow E \rightarrow PG \rightarrow \text{Job}$

$T \rightarrow P \rightarrow E \rightarrow PG \rightarrow \text{Job}$

$T \rightarrow P \rightarrow \text{deg} \rightarrow PG \rightarrow \text{Business}$

\* connectivity :- It is the process of visiting the possible vertices in directed graph.

⇒ This concept can be presented the following graph



⇒ for the above graph, draw the connectivity table.

⇒ In this, the first letter is starting vertex and next letters are reached vertices directly or through other vertices.

Table : AB → if start with A, we reached B.

CDB → if start with C, we reached B through D.

DB → if start with D, we reached B.

-B → if start with B, we reached no vertex.

Adjacency matrix for the above graph is :-

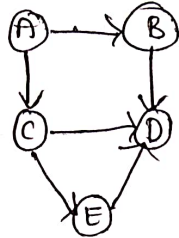
|   | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 |
| B | 0 | 0 | 0 | 0 |
| C | 0 | 1 | 0 | 1 |
| D | 0 | 1 | 0 | 0 |

\*\*

Q:- Write about warshall's Algorithm (Transitive closure property in Graphs)

Ans:- warshall's Algorithm:- This algorithm includes the process like if we visit from A to B and from B to C, Automatically we can visit, from A to C

Ex:-



⇒ In the above directed graph the adjacency matrix will be

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 | 0 |
| B | 0 | 0 | 0 | 1 | 0 |
| C | 0 | 0 | 0 | 1 | 1 |
| D | 0 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 1 | 0 |

⇒ If we apply warshall's Algorithm, the above adjacency matrix be changed as shown below:-

→ The transitive closure can be presented as:

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 1 |
| B | 0 | 0 | 0 | 1 | 0 |
| C | 0 | 0 | 0 | 1 | 1 |
| D | 0 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 1 | 0 |

changed due to warshall's Algorithm.

Here, this concept is useful to satisfy the process of connectivity.



## Q Define Graph & Array and linked list Representation of Graphs ⑦

Q) Explain different, Representation Techniques of Graphs

Ans :- Graph :- It is non-linear datastructure and combination of vertices and edges.

⇒ Normally it is not possible to represent a graph in physical manner.

⇒ so, we represent the Graphs in logical manner

Representation of Graphs :-

for the logical Representation of Graphs, we use two techniques They are

① Adjacency matrix (Array Representation)

② Adjacency list (linkedlist Representation)

① Adjacency matrix :-

→ It is one of the representation technique for presenting a graph in logical manner.

→ For this technique, it is required to develop a matrix by using labels of vertices 0 & 1

→ When we develop Adjacency matrix, we should follow below steps:

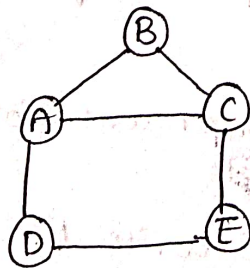
⇒ Take vertex labels as Row headings & column headings

⇒ The values of matrix are filled by 0's and 1's (Zero's and one's)

⇒ If there are 2 vertices are adjacent to each other, then the matrix value will be '1' otherwise '0'.

⇒ This process can be represented with the following Example graphs:

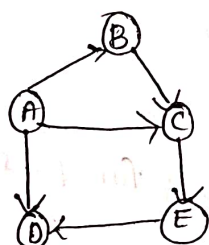
\* Non-directed graph :-



Adjacency matrix of above undirected graph :-

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 |
| B | 1 | 0 | 1 | 0 | 0 |
| C | 1 | 1 | 0 | 0 | 1 |
| D | 1 | 0 | 0 | 0 | 1 |
| E | 0 | 0 | 1 | 1 | 0 |

\* Directed graph :-



Adjacency matrix of above directed graph :-

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 |
| B | 0 | 0 | 1 | 0 | 0 |
| C | 0 | 0 | 0 | 0 | 1 |
| D | 0 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 1 | 0 |

② Adjacency list :-

⇒ It is one of the technique for presenting a graph in logical manner.

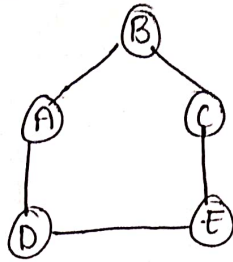
⇒ This technique uses linked list representation for presenting graph

⇒ In this technique, we need to write list of adjacent vertices of each vertex.



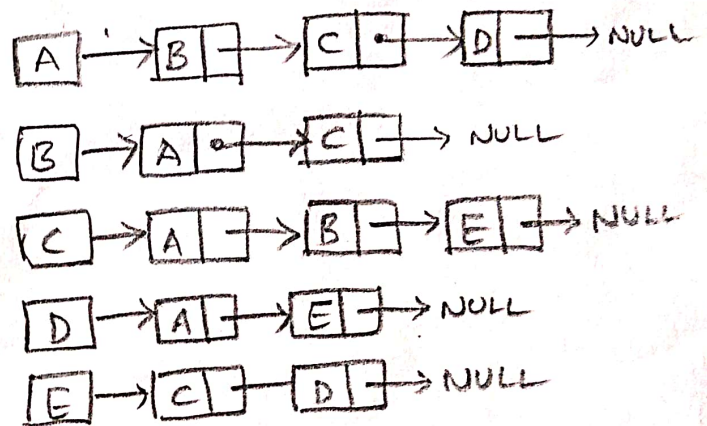
⇒ This will be represented with the following Example graphs: 5

\* Non-directed graph :-

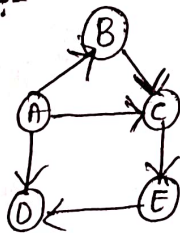


Adjacency list for above Non-Directed graph is as shown below:

| <u>Vertices</u> | <u>list</u> |
|-----------------|-------------|
| A               | B → C → D   |
| B               | A → C       |
| C               | A → B → E   |
| D               | A → E       |
| E               | C → D       |

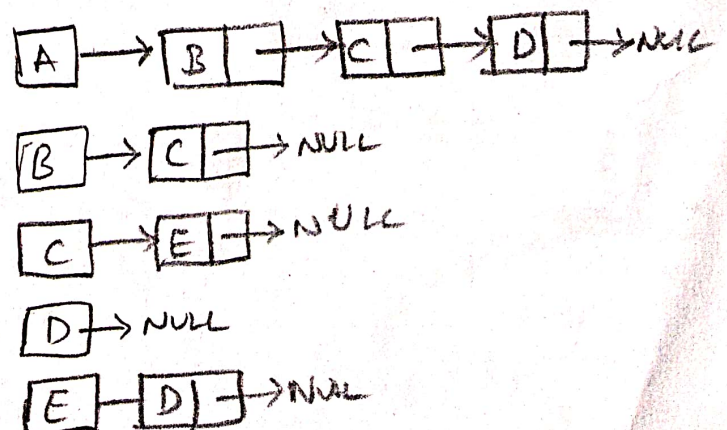


Directed graph :-



Adjacency list for above Directed Graph is as follows:-

| <u>vertices</u> | <u>list</u>                                      |
|-----------------|--------------------------------------------------|
| A               | B → C → D                                        |
| B               | C                                                |
| C               | E                                                |
| D               | (since D have no direction to any other vertex). |
| E               | D                                                |



## Graph Traversal (Searching) Algorithms :: DFS and BFS

Graph traversal is a process of visiting all vertices in the graph and also search technique for finding a vertex in a graph.

In the search process, graph traversal is also used to determine the order in which it visits the vertices. There are two methods to traverse a graph data structure:

1. Depth-First Search ( DFS ) Algorithm
2. Breadth-First Search ( BFS ) Algorithm

These searching algorithms produces a " **Spanning Tree** " as final result which represents the traversing order.

**Spanning Tree** : It is the sub graph from given graph with minimum no. of Edges and satisfies the following Conditions :

- a) All Vertices are included and Cycles should not formed
- b) Number of Edges = Number of Vertices – 1

### \*\*Depth-First Search (DFS) Algorithm (or)

### \*\* (Back Tracking - Application of Stack -Unit-3)

DFS starts at a source node and explores as deep as possible along each path before backtracking.

To implement DFS traversal, we use stack data structure

#### Algorithm :

**Step 1:** Create a stack with the total number of vertices in the graph.

**Step 2:** Choose any vertex as the traversal's start point. Visit that vertex and Push it into stack.

**Step 3 :** Visit and Push non-visited adjacent vertices of a vertex which is at the top of the stack

**Step 4 :** Repeat step 3 until there are no more adjacent vertices to for the vertex at the top of the stack.

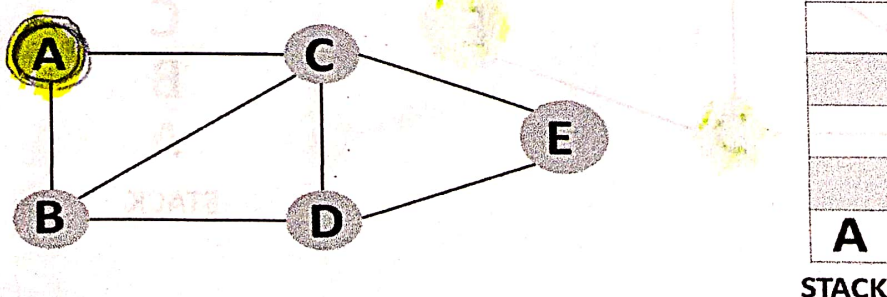
**Step 5 -** If there are no new vertices to visit, go back and pop one from the stack using **backtracking**.

**Step 6 -** Continue using steps 3, 4, and 5 until the stack is empty.

**Step 7 -** When the stack is Empty, create the final spanning tree by deleting the graph's unused edges.

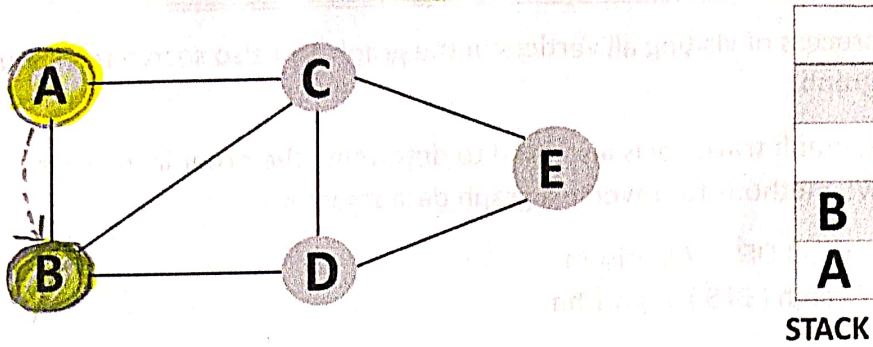
#### Example for DFS Algorithm :

**Step 1:** Mark vertex A as a visited source node(start) and push vertex A to the top of the stack.

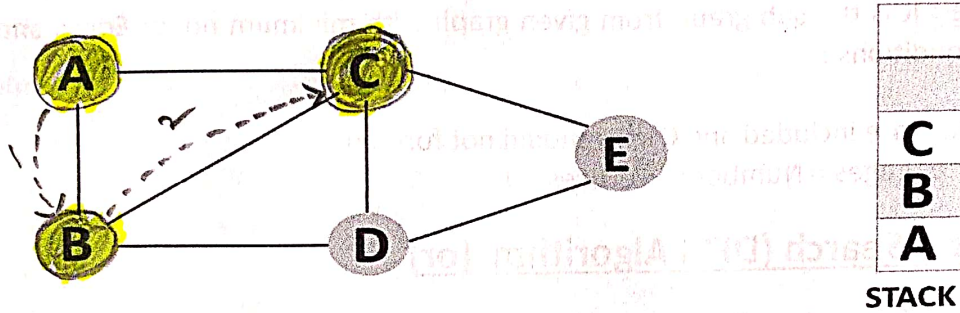




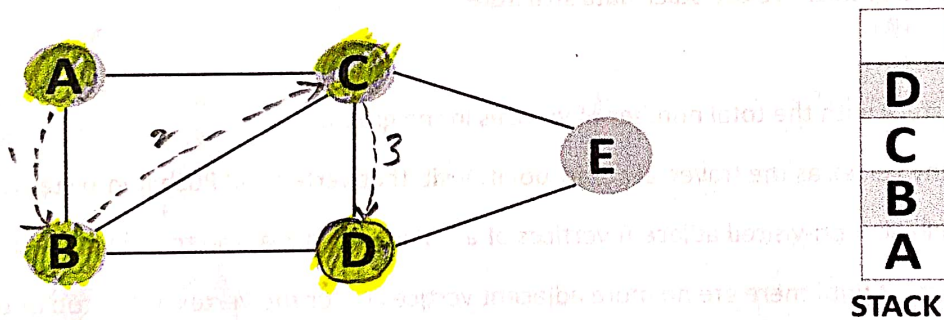
**Step 2:** unvisited vertices of A are B and C, let we choose B, so visit and push vertex B into top of stack.



**Step 3:** From B vertex, C and D are adjacent unvisited vertices. choose C, visit and push C into stack

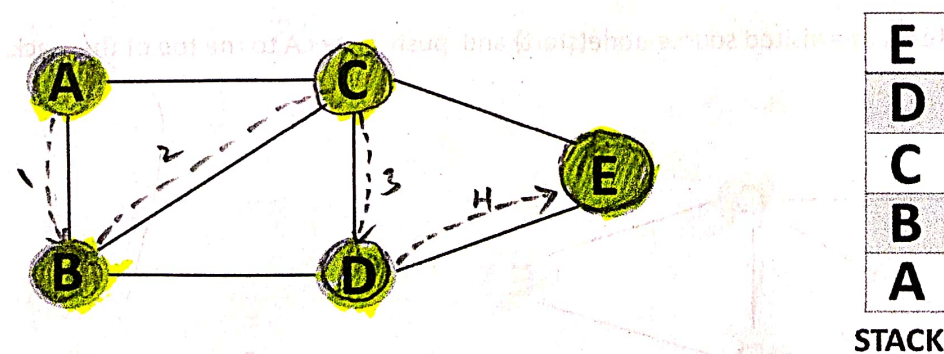


**Step 4:** From vertex C, D and E are adjacent unvisited. select vertex D visit and push vertex D into stack.

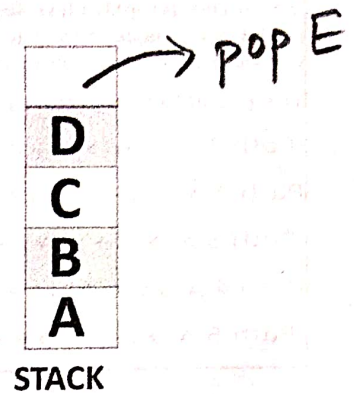
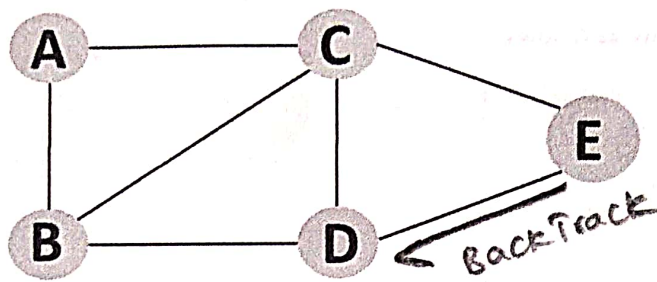


**Step 5:** Vertex E is the lone unvisited adjacent vertex of vertex D, thus marking it as visited.

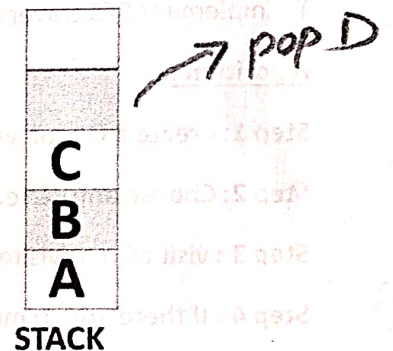
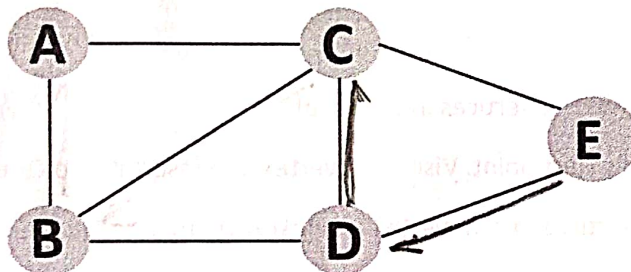
Vertex E should be pushed to the top of the stack.



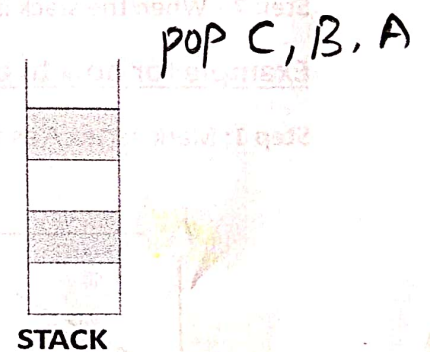
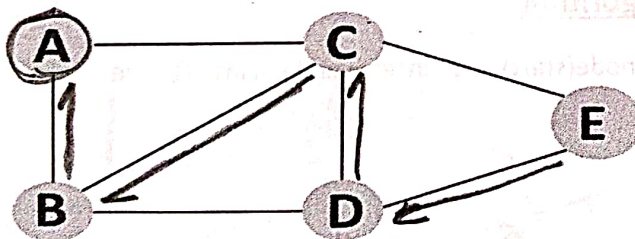
Step 6: vertex E has no unvisited neighbours  $\rightarrow$  Backtrack to D . For this pop vertex E from the stack.



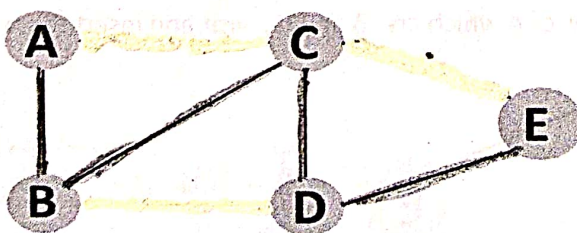
Step 7: Since D has no more adjacent , Backtrack to C . pop D from stack



Step 8: Similarly C , B and finally A also have no unvisited ,back track to C $\rightarrow$ B $\rightarrow$ A .pop them from the stack. As result Stack become Empty.



Since Stack is Empty , stop The traversing and we get the path and Spanning Tree as shown below



path: A - B - C - D - E



Note :

Since **DFS (Depth-First Search)** depends on the order in which neighbours are visited, **multiple DFS paths are possible** for the same graph — as long as they follow the core DFS rule: *go as deep as possible before backtracking*.

The possible DFS paths for above graph are as follows

Path 1:  $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$  ✓ *we class get*

Path 2:  $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E$

Path 3:  $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

Path 4:  $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E$

Path 5:  $A \rightarrow C \rightarrow D \rightarrow B \rightarrow E$

## 2) \*\*\* BFS (Breadth First Search) : (Application of Queue -Unit- 3 )

BFS starts at a source node and explores as deep as possible along each path before backtracking.

To implement BFS traversal, we use stack data structure

### Algorithm :

**Step 1:** Create a Queue with size total number of vertices in the graph.

**Step 2:** Choose any vertex as the traversal's start point. Visit that vertex and insert it into Queue.

**Step 3 :** visit all non-visited and adjacent vertices of that vertex and Insert them into Queue.

**Step 4 :** If there are no more adjacent vertices, Remove a vertex from Queue.

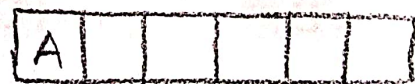
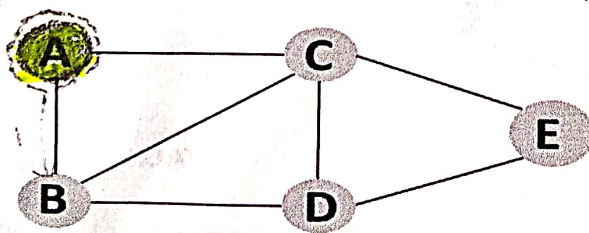
**Step 5 :** Repeat step-3 from removed vertex.

**step 6 -** Repeat steps 3, 4, and 5 until the Queue is empty.

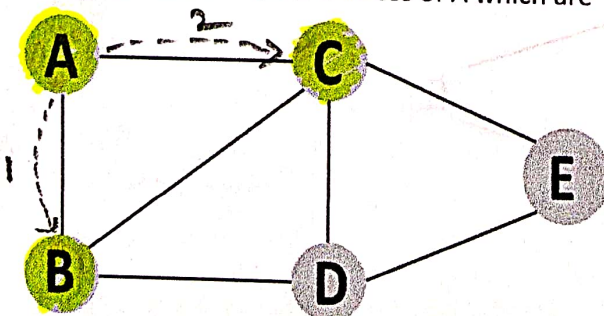
**Step 7 -** When the stack is Empty, create the final spanning tree by deleting the graph's unused edges.

### Example for how to use the BFS algorithm .

**Step 1:** Mark vertex A as a visited source node(start) and insert vertex A into Queue.



**Step 2:** visit all adjacent and unvisited vertices of A which are B and C , visit and insert them into

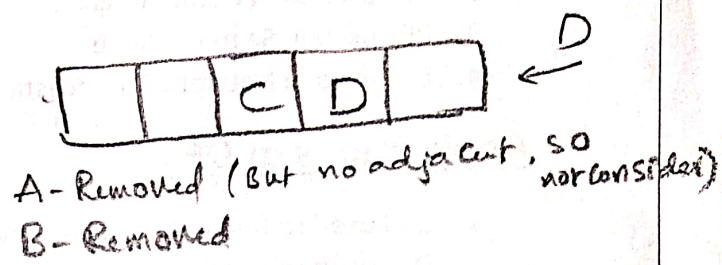
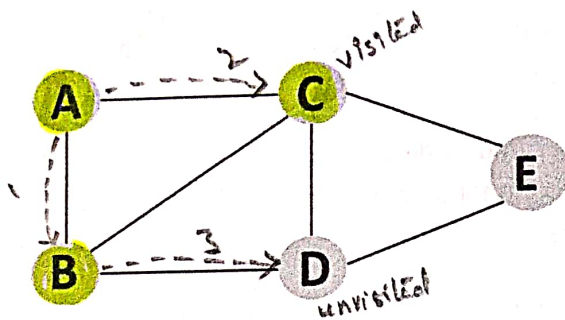


Queue.

**Step 3:** Now A has not adjacent, Remove vertex from Queue. And repeat above process from deleted vertex.

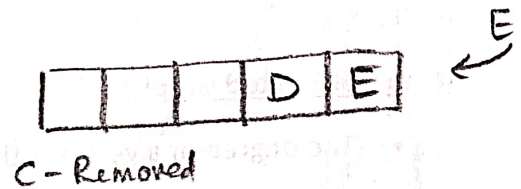
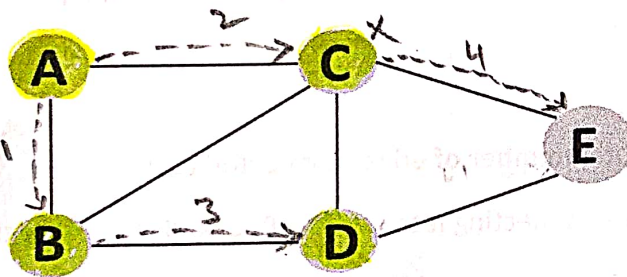
Removed vertex is B, So from B, Visit adjacents (C, D) but C is already visit.

Then visit D and insert into Queue.



**Step 4:** Now there are no adjacent for B, So remove a vertex from Queue which is C.

Then Visit adjacent of C which are D and E, But D is already visited, so visit E and insert into Queue

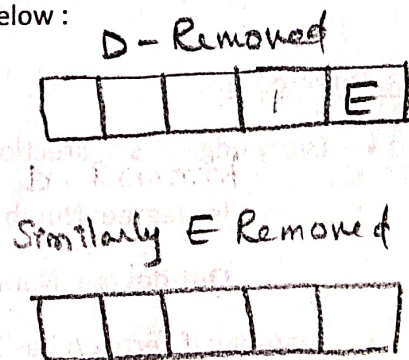
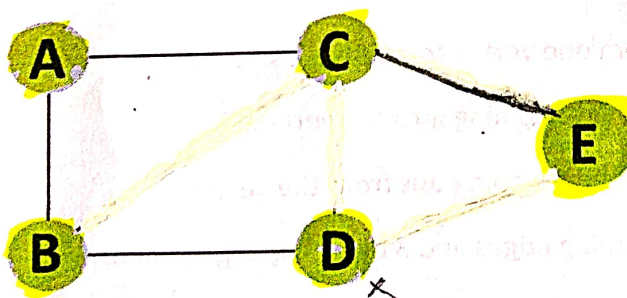


**Step 5:** Now there are no adjacent for C, So remove a vertex from Queue which is D.

Then Visit adjacent of D which are B, D and E, But B, D and E are already visited, So Remove E

Now Queue become Empty So stop the traverse.

Finally the path and Spanning tree are formed as shown below :



path:

- A → B, C
- B → D
- C → E
- D → NO unvisited
- C → NO unvisited

**Note :** BFS also have multiple possible alternative paths based on start vertex, selection of adjacent, etc



1. BFS used to create shortest paths and minimum Spanning Trees.
2. BFS used in P2P Networks.
3. BFS used in Search Engines.
4. BFS used in Network Broadcasting.

## Applications of DFS :

1. DFS used to find path between two specified nodes.
2. DFS used to check whether a Graph is connected or not.
3. DFS used to Computing Spanning Tree

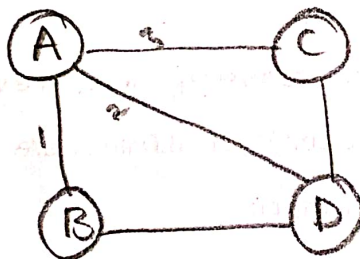
---

## Degree of Node (or) Vertex :

The **degree** of a **node (or vertex)** in a graph refers to the number of edges connected to that node.

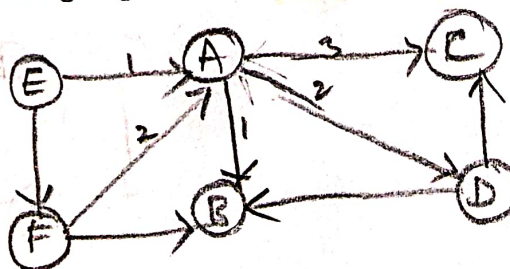
### In an UnDirected Graph:

- The **degree** of a vertex is the **total number of edges** connected to it.
- Example: If vertex A has 3 edges connecting it to vertices B, C, and D, then  $\deg(A) = 3$ .



### In a Directed Graph:

- Every edge has a direction (from one vertex to another), so we have:
  - **In-degree:** Number of edges coming **into** the vertex.
  - **Out-degree:** Number of edges going **out from** the vertex.
- Example: If vertex A has 2 incoming edges and 3 outgoing edges, then:



- $\text{in-degree}(A) = 2$
- $\text{out-degree}(A) = 3$

## \*\*\* Hashing, Hashing Mechanism , Hash Functions

➔ **Hashing** is a well-known technique to search any particular element among several elements.

It minimizes the number of comparisons while performing the search.

Hashing is extremely efficient.

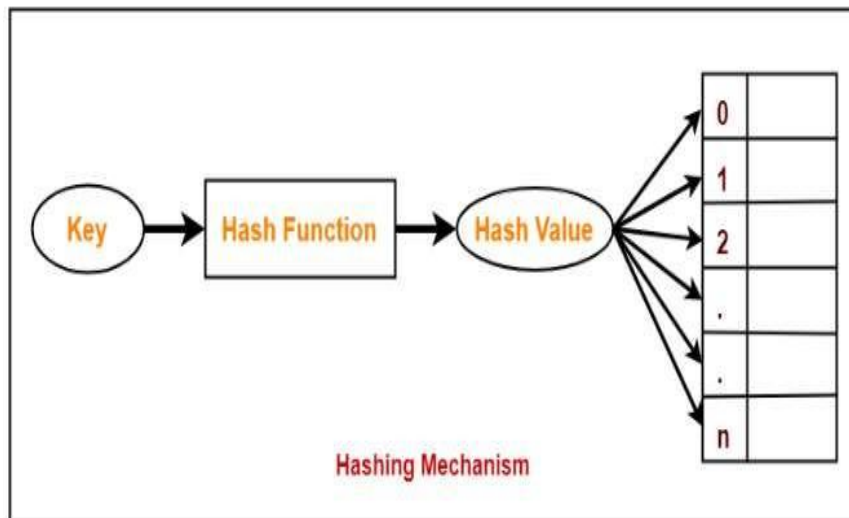
The time taken by it to perform the search does not depend upon the total number of elements like Linear and Binary Search.

It completes the search with constant time complexity  $O(1)$ .

➔ **Hashing Mechanism:** An array data structure called as **Hash table** is used to store the data items, Based on the hash key value, data items are inserted into the hash table.

**Hash key value** is a special value that serves as an index for a data item.

- It indicates where the data item should be stored in the hash table.
- Hash key value is generated using a hash function.





## Hash Function

- **Hash function** is a function that maps any big number or string to a small integer value.
- Hash function takes the data item as an input and returns a small integer value as an output.
- The small integer value is called as a hash value.
- Hash value of the data item is then used as an index for storing it into the hash table.

### Types of Hash Functions:

There are various types of hash functions available such as-

**1.Division Hash Function** : Uses modulo to keep the hash within table size.

$$h(\text{key}) = \text{key} \% \text{tableSize}$$

$$\text{Ex : } h(46) = 46 \% 7 = 4$$

**2.Mid Square Hash Function** : Square the key and extract the middle digits.

Example: Key = 123  $\rightarrow 123^2 = 15129 \rightarrow$  Take middle digits 512  $\rightarrow 512 \% \text{tableSize}$

**3.Folding Hash Function** : Breaks the key into parts, adds them together.

Example: For key = 12345678  $\rightarrow$  Split into 1234 + 5678 = 6912  $\rightarrow h(\text{key}) = 6912 \% \text{tableSize}$

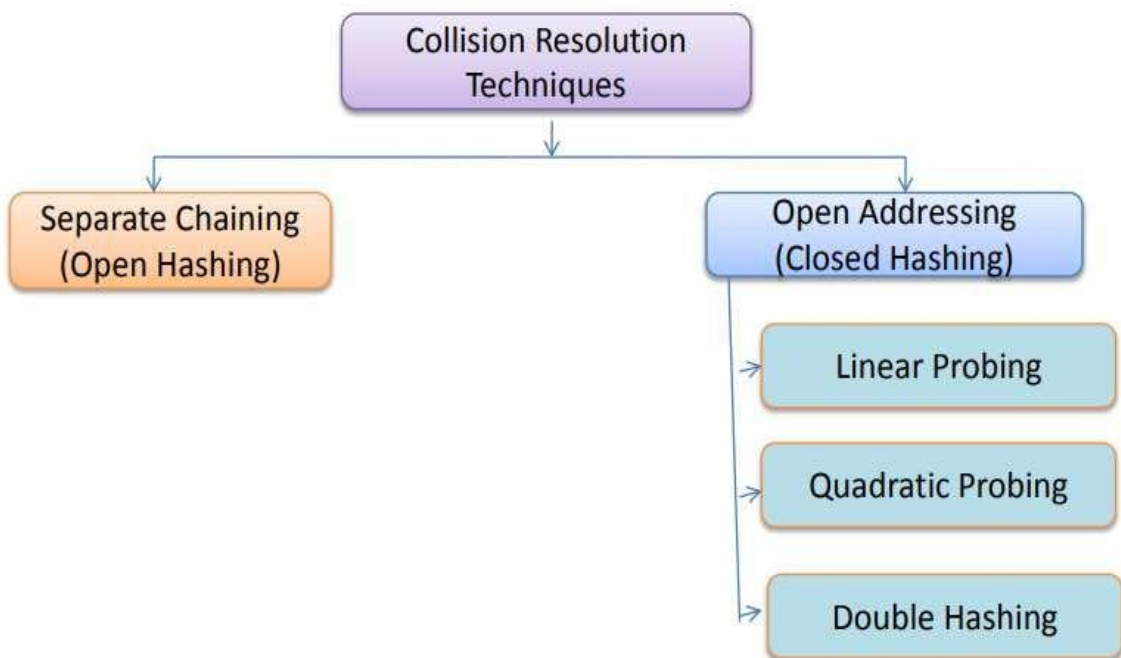
**Properties of Hash Function:** The properties of a good hash function are-

- It is efficiently computable.
  - It minimizes the number of collisions.
  - It distributes the keys uniformly
-

## **\*\* Collision in Hashing ( Collision Resolution Techniques )**

- ✓ Hash function is used to compute the hash value for a key.
- ✓ Hash value is then used as an index to store the key in the hash table.
- ✓ Hash function may return the same hash value for two or more keys.
- ✓ When the hash value of a key maps to an already occupied bucket of the hash table, it is called as **a Collision**.

**Collision Resolution Techniques:** Collision Resolution Techniques are the techniques used for resolving or handling the collision.





## **\*\*Separate Chaining (Open Chaining/ Open Hashing )**

**To handle the collision,** This technique creates a linked list to the slot for which collision occurs,

- The new key is then inserted in the linked list.
- These linked lists to the slots appear like chains.
- That is why, this technique is called as separate chaining.

### **Example for Separate Chaining:**

• Using the hash function '**key mod 7**', insert the following sequence of keys in the hash table

50, 700, 76, 85, 92, 73 and 101

#### **Step-1**

- ✓ Draw an empty hash table.
- ✓ For the given hash function, the possible range of hash values is  $[0, 6]$ . So, draw an empty hash table consisting of 7 buckets as shown -

|   |  |
|---|--|
| 0 |  |
| 1 |  |
| 2 |  |
| 3 |  |
| 4 |  |
| 5 |  |
| 6 |  |

#### **Step-2**

- ✓ Insert the given keys in the hash table one by one.
- ✓ The first key to be inserted in the hash table = 50.
- ✓ Bucket of the hash table to which key 50 maps =  $50 \bmod 7 = 1$ .
- ✓ So, key 50 will be inserted in bucket-1 of the hash table as-

|   |    |
|---|----|
| 0 |    |
| 1 | 50 |
| 2 |    |
| 3 |    |
| 4 |    |
| 5 |    |
| 6 |    |

#### **Step-3**

- ✓ The next key to be inserted in the hash table = 700.
- ✓ Bucket of the hash table to which key 700 maps =  $700 \bmod 7 = 0$ .
- ✓ So, key 700 will be inserted in bucket-0 of the hash table as-

|   |     |
|---|-----|
| 0 | 700 |
| 1 | 50  |
| 2 |     |
| 3 |     |
| 4 |     |
| 5 |     |
| 6 |     |

**Step-4**

The next key to be inserted in the hash table = 76.

Bucket of the hash table to which key 76 maps =  $76 \bmod 7 = 6$ .

So, key 76 will be inserted in bucket-6 of the hash table as-

|   |     |
|---|-----|
| 0 | 700 |
| 1 | 50  |
| 2 |     |
| 3 |     |
| 4 |     |
| 5 |     |
| 6 | 76  |

**Step-5**

The next key to be inserted in the hash table = 85.

Bucket of the hash table to which key 85 maps =  $85 \bmod 7 = 1$ .

Since bucket-1 is already occupied, so collision occurs.

Separate chaining handles the collision by creating a linked list to bucket-1.

So, key 85 will be inserted in bucket-1 of the hash table as-

|   |     |      |
|---|-----|------|
| 0 | 700 |      |
| 1 | 50  | → 85 |
| 2 |     |      |
| 3 |     |      |
| 4 |     |      |
| 5 |     |      |
| 6 | 76  |      |

**Step-6**

The next key to be inserted in the hash table = 92.

Bucket of the hash table to which key 92 maps =  $92 \bmod 7 = 1$ .

Since bucket-1 is already occupied, so collision occurs.

Separate chaining handles the collision by creating a linked list to bucket-1.

So, key 92 will be inserted in bucket-1 of the hash table

|   |     |           |  |
|---|-----|-----------|--|
| 0 | 700 |           |  |
| 1 | 50  | → 85 → 92 |  |
| 2 |     |           |  |
| 3 |     |           |  |
| 4 |     |           |  |
| 5 |     |           |  |
| 6 | 76  |           |  |

**Step-7**

The next key to be inserted in the hash table = 101.

Bucket of the hash table to which key 101 maps =  $101 \bmod 7 = 3$ .

Since bucket-3 is already occupied, so collision occurs.

Separate chaining handles the collision by creating a linked list to bucket-3.

So, key 101 will be inserted in bucket-3 of the hash table as

|   |     |           |  |
|---|-----|-----------|--|
| 0 | 700 |           |  |
| 1 | 50  | → 85 → 92 |  |
| 2 |     |           |  |
| 3 | 73  | → 101     |  |
| 4 |     |           |  |
| 5 |     |           |  |
| 6 | 76  |           |  |

## **\*\*Open Addressing ( Closed Hashing )**

Unlike separate chaining, all the keys are stored inside the hash table and No key is stored outside the hash table.

• Techniques used for open addressing are-

1. Linear Probing
2. Quadratic Probing
3. Double Hashing

### **Operations in Open Addressing:**

#### **i) Insert Operation:**

- Hash function is used to compute the hash value for a key to be inserted.
- Hash value is then used as an index to store the key in the hash table.

#### **In case of collision,**

- Probing is performed until an empty bucket is found.
- Once an empty bucket is found, the key is inserted.
- Probing is performed in accordance with the technique used for open addressing.

#### **ii) Search Operation:**

To search any particular key, we follow the below steps :

1. Its hash value is obtained using the hash function used.
2. Using the hash value, that bucket of the hash table is checked.
3. If the required key is found, the key is searched.
4. Otherwise, the subsequent buckets are checked until the required key or an empty bucket is found.
5. The empty bucket indicates that the key is not present in the hash table.

#### **iii )Delete Operation:**

The key is first searched and then deleted. After deleting the key, that particular bucket is marked as "deleted".



- So insert 47 in bucket index 0 .

## 2. Quadratic Probing

In quadratic probing, When collision occurs, we probe for  $i^2$  th bucket in  $i^{\text{th}}$  iteration.

$$(h(k) + i^2) \bmod \text{SIZE} \quad (\text{here } i=0,1,2,\dots)$$

-We apply above probing until an empty bucket is found.

### Example for Quadratic Probing

Probe sequence is

- $h(k) \bmod \text{size}$
- $(h(k) + 1) \bmod \text{size}$
- $(h(k) + 4) \bmod \text{size}$
- $(h(k) + 9) \bmod \text{size}$
- ...

|                             |                             |                             |
|-----------------------------|-----------------------------|-----------------------------|
| insert(76)<br>$76 \% 7 = 6$ | insert(40)<br>$40 \% 7 = 5$ | insert(48)<br>$48 \% 7 = 6$ |
| 0                           | 0                           | 0                           |
| 1                           | 1                           | 1                           |
| 2                           | 2                           | 2                           |
| 3                           | 3                           | 3                           |
| 4                           | 4                           | 4                           |
| 5                           | 5                           | 5                           |
| 6                           | 6                           | 6                           |
| 76                          | 40                          | 48                          |

In the above example, 76, 40, are inserted as per regular Hash function.

But when we want to insert 48, Collision occurs because  $48 \% 7 = 6$ , bucket 6 is already filled

$$\begin{aligned}
 \text{Then probe sequence is} &= (H(k) + 1) \bmod \text{SIZE} \\
 &= (48 + 1) \bmod 7 \\
 &= 0
 \end{aligned}$$

So insert 48 at bucket index 0 as shown above.

### Advantage :

- Reduces Primary Clustering
- Simple to Implement

### Disadvantage :

- Secondary Clustering
- May Not Find Empty Slot If Table Not Properly Sized





Probing sequence (index to Insert) =  $h_1(k) + 1 * h_2(k)$

$$= 5 + 1 \times 6$$

$$= 9$$

$$= 9 \bmod 7$$

$$= 2 \text{ (already filled and still collision)}$$

- Since still collision, try to insert at index =  $5 + 2 \times 6 = 16$

$$= 17$$

$$= 17 \bmod 7$$

$$= 3 \text{ (empty, no collision)}$$

Finally insert 47 at index "3" as shown below : **insert (47)**

|           |   |
|-----------|---|
|           | 0 |
|           | 1 |
| <b>93</b> | 2 |
| <b>47</b> | 3 |
|           | 4 |
| <b>40</b> | 5 |
| <b>76</b> | 6 |

## \*\*Separate chaining Vs Open Addressing

| Separate Chaining                                                                                          | Open Addressing                                                                                  |
|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Keys are stored inside the hash table as well as outside the hash table.                                   | All the keys are stored only inside the hash table.<br>No key is present outside the hash table. |
| The number of keys to be stored in the hash table can even exceed the size of the hash table.              | The number of keys to be stored in the hash table can never exceed the size of the hash table.   |
| Deletion is easier.                                                                                        | Deletion is difficult.                                                                           |
| Extra space is required for the pointers to store the keys outside the hash table.                         | No extra space is required.                                                                      |
| Cache performance is poor.<br>This is because of linked lists which store the keys outside the hash table. | Cache performance is better.<br>This is because here no linked lists are used.                   |
| Some buckets of the hash table are never used which leads to wastage of space.                             | Buckets may be used even if no key maps to those particular buckets.                             |